

Rapport technique

Optimiser l'orientation des patients et soutenir le personnel soignant aux urgences

Spécialisation d'un modèle de base par Fine-Tuning Supervisé (SFT) et alignement par préférences pour le déploiement d'un LLM médical au service de la santé publique.

por **Rafael CEREZO MARTIN**
AI Engineer Openclassrooms
Septiembre 2026

Tabla de Contenidos	
Introduction	1
Contraintes techniques imposées	2
Architecture retenue	2
Concepts clés	3
Environnement de développement	4
1. Préparation des données	4
1.1 Méthodologie	4
1.2 Anonymisation et contrôle qualité	4
1.3 Résultat	5
1.4 Baseline zero-shot	5
2. SFT + LoRA	5
2.1 Méthodologie	5
2.2 Résultat	5
2.3 Evaluation Post-SFT	5
3. DPO (Direct Preference Optimization)	6
3.1 Méthodologie	6
3.2 Métriques et itérations réelles	6
3.3 Évaluation post-DPO	6
3.4 Portée de la reformulation des préférences	6
3.5 Tentative de mitigation à l'inférence	6
4. Déploiement	7
4.1 Architecture retenue	7
4.2 Incident technique majeur et résolution	8
4.3 CI/CD et endpoint réel	8
4.4 Traçabilité et sécurité applicative	9
5. Sécurité, conformité et vie privée	9
6. Analyse des résultats	9
7. Roadmap et recommandations pour le passage à l'échelle	10
7.1 Court terme (avant toute extension de périmètre)	10
7.2 Moyen terme (fiabilisation avant extension d'usage)	11
7.3 Long terme (déploiement à l'échelle du CHSA)	11
Conclusion	11
Références	11

Glossaire

Terme	Définition
DPO	Direct Preference Optimization , alignement à partir de paires (réponse préférée, réponse rejetée).
ESI	Emergency Severity Index , échelle de triage aux urgences, niveaux 1 (critique) à 5 (non urgent).
LoRA	Low-Rank Adaptation , fine-tuning par matrices de rang réduit, modèle de base gelé.
NF4 (quantification)	NormalFloat 4-bit , format de quantification des poids, distinct du garde-fou de sécurité clinique homonyme.
PagedAttention	Gestion par pages du cache d'attention GPU dans vLLM, comme une mémoire virtuelle.
POC	Proof of Concept , démonstrateur de faisabilité, pas un produit de production.
QLoRA	LoRA appliqué sur un modèle chargé en quantification 4-bit.
SFT	Supervised Fine-Tuning , spécialisation supervisée classique, exemple par exemple.
vLLM	Moteur de service d'inférence LLM haute performance, imposé par le cahier des charges.

Tableau , Glossaire des sigles et termes techniques utilisés dans ce rapport.

Abstract

Ce rapport documente la conduite d'un projet de type proof of concept (POC) pour le Centre Hospitalier Saint-Aurélien (CHSA) : la spécialisation d'un modèle de langage de base (Qwen3-1.7B-Base) par **fine-tuning supervisé (SFT)** puis **alignement par préférences (DPO)**, afin d'assister le personnel infirmier lors de l'accueil aux urgences par un court entretien guidé et une proposition de niveau de priorité de triage (échelle ESI), systématiquement soumise à validation humaine. Le projet suit une architecture hexagonale (ports/adaptateurs) découplant le domaine métier de la technologie d'inférence, ce qui a permis de faire fonctionner la même logique d'entretien indifféremment contre trois moteurs distincts (local, distant, double de test) tout au long du pipeline, de la préparation et l'anonymisation des données jusqu'au déploiement d'un endpoint d'inférence réel sur vLLM.

Les résultats mesurés sur un même jeu de test de 278 exemples montrent un effet net du SFT : le F1 moyen (token) passe de 0,043 (meilleure baseline, GPU bf16) à 0,112, un score quasiment triplé. L'entraînement DPO qui a suivi a d'abord provoqué une régression inattendue (F1 retombé à 0,049, dégénérescence du modèle en langue et en répétitions), diagnostiquée comme un ancrage insuffisant au modèle de référence et corrigée par un ajustement de l'hyperparamètre beta ($0,1 \rightarrow 0,3$) ; ce correctif a été confirmé à l'échelle réelle de 5000 paires de préférence, avec un F1 stabilisé à 0,110, sans régression au passage à l'échelle. Le pipeline complet a été déployé sur un endpoint réel (conteneur Docker GPU sur Hugging Face Space, pipeline CI/CD vert), avec traçabilité systématique des échanges. Le POC démontre ainsi la faisabilité technique de bout en bout, tout en laissant clairement hors périmètre la validation médicale et réglementaire nécessaire à tout déploiement clinique réel.

Introduction

Contexte et mission

Le Centre Hospitalier Saint-Aurélien (CHSA) souhaite évaluer l'apport d'un agent conversationnel fondé sur un grand modèle de langage (LLM) pour assister le personnel infirmier lors de l'accueil aux urgences : recueillir les symptômes d'un patient par un court entretien guidé, puis proposer un niveau de priorité de triage (échelle ESI, 1 à 5) à valider par un soignant humain. Ce rapport documente le travail réalisé dans le cadre de cette mission de type proof of concept (POC) : préparation des données, spécialisation du modèle par SFT puis DPO, et déploiement d'un endpoint d'inférence réel.

L'agent n'a volontairement aucun pouvoir de décision clinique autonome : il recueille l'information et propose une classification, mais chaque diagnostic reste soumis à validation par un infirmier avant toute action. Cette limite de périmètre, posée dès le départ, structure une grande partie des choix techniques décrits dans ce rapport (traçabilité systématique, garde-fous de sécurité, format structuré vérifiable).

Pour comprendre l'enjeu, il faut rappeler ce qu'est concrètement un **triage aux urgences** : à l'arrivée d'un patient, l'infirmier d'accueil dispose de quelques minutes pour évaluer la gravité de la situation et décider qui doit être vu en priorité. L'échelle **ESI (Emergency Severity Index)** sert de repère commun pour cette décision, du niveau 1 (urgence vitale immédiate) au niveau 5 (consultation non urgente). C'est un exercice difficile sous contrainte de temps, et c'est précisément là que se situe l'apport potentiel de l'agent : préparer le terrain en recueillant l'information de façon structurée, pour que la décision finale, elle, reste entièrement entre les mains du soignant.

Contraintes techniques imposées

Le cahier des charges impose un modèle de base **Qwen3-1.7B-Base** spécialisé par **LoRA** (SFT puis DPO), un moteur d'inférence **vLLM** (PagedAttention), une API **FastAPI** conteneurisée **Docker**, une anonymisation par **Microsoft Presidio** et un pipeline **CI/CD** GitHub Actions. Ces choix ne sont pas discutés ici : ce rapport porte sur leur mise en œuvre et sur les résultats obtenus.

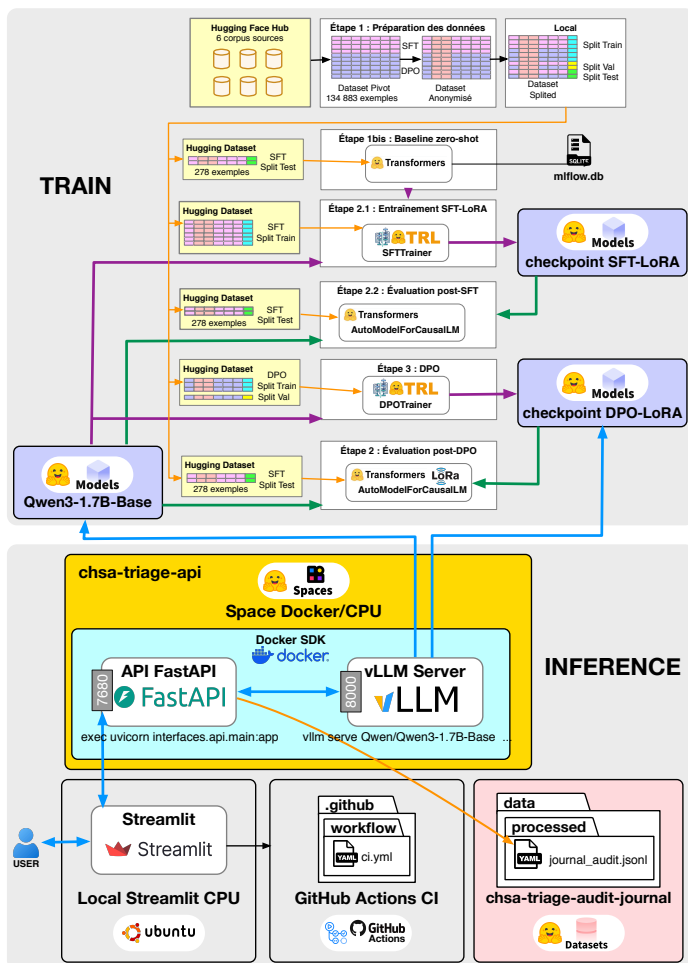


Figure. Deux chaînes distinctes imposées par le cahier des charges : entraînement (SFT puis DPO du modèle de base) et inférence (service du checkpoint via vLLM)

Ces deux chaînes restent volontairement séparées : **l'entraînement** produit un checkpoint versionné sur le Hub, **l'inférence** le sert ensuite tel quel, sans jamais les confondre dans un seul pipeline.

Architecture retenue

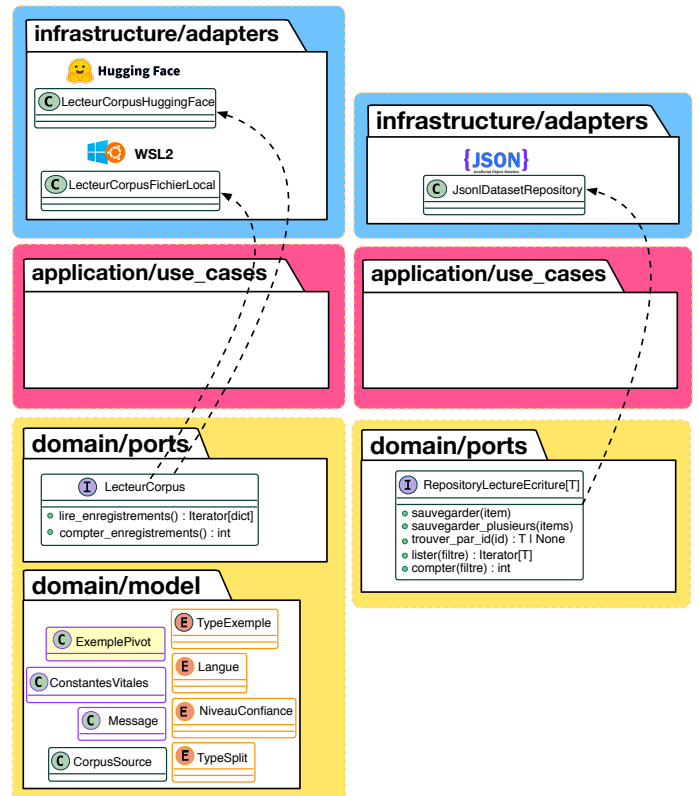
Le projet suit une **architecture hexagonale** (ports/adaptateurs) :

Le **domaine métier** (cas d'usage d'entretien et de diagnostic) ignore totalement la technologie d'inférence, de persistance ou d'anonymisation utilisée.

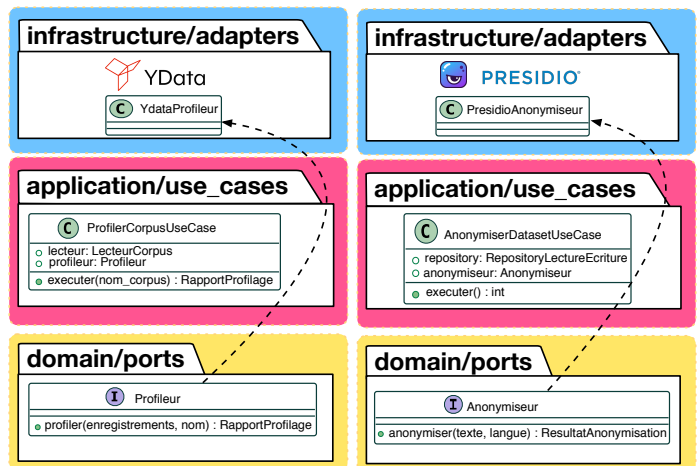
Ce choix s'est révélé décisif pendant le déploiement (§4) : la même logique d'entretien fonctionne indifféremment contre un moteur local (llama.cpp), un moteur distant (vLLM) ou un double en mémoire dans les tests, sans aucune modification du code métier.

Concrètement, le domaine définit un port **« MoteurInference »** (une interface abstraite : « générer une réponse à partir de messages ») dont trois adaptateurs distincts implémentent chacun un moyen technique différent d'y répondre, c'est ce découplage qui a permis, pendant l'incident de déploiement (§4.2), de continuer à développer et tester le reste du système sans dépendre en permanence d'un GPU réel disponible.

Pour le dire simplement : le domaine métier pose une question générique (« génère une réponse à partir de cette conversation ») sans jamais savoir qui va y répondre concrètement. C'est un peu comme une prise électrique standard, n'importe quel appareil compatible peut s'y brancher, sans que la prise elle-même ait besoin de connaître l'appareil à l'avance. Le domaine se contente de brancher un adaptateur ou un autre selon le contexte (test, développement local, production), sans jamais changer sa propre logique.



Figure, Exemples d'adaptateurs Génériques



Figure, Exemples d'adaptateurs dédiés à l'implémentation des Cas d'Utilisation

Tabla 1 : Adaptateurs principaux :

Adaptateur	Rôle
LlamaCppInferenceAdapter	Moteur local CPU (llama.cpp), sans GPU
TransformersInferenceAdapter	Moteur GPU distant (HF Jobs), modèle de base
TransformersLoraInferenceAdapter	Idem, avec adaptateur LoRA chargé
VllmEndpointInferenceAdapter	Moteur réellement déployé en production (vLLM)
ChatMLFormateurAdapter	Mise en forme des messages au format ChatML.
TrlSftEntraîneurAdapter	Pilotage réel de l'entraînement SFT (bibliothèque TRL)
TrlDpoEntraîneurAdapter	Pilotage réel de l'entraînement DPO (bibliothèque TRL)

Tabla 2 : Cas d'usage principaux, par étape

Cas d'usage	Rôle
Étape 1 : Préparation des données	
ProfilerCorpusUseCase	Profiler un corpus brut avant nettoyage
ConstruireDatasetPivotUseCase	Convertir une source brute vers le schéma pivot commun
AnonymiserDatasetUseCase	Anonymiser les champs texte libre (Presidio)
ReviserPiiResiduelleUseCase	Révision humaine des cas de PII résiduelle douteux
ControlerQualiteAnonymisationUseCase	Contrôle qualité RGPD, pivot original vs anonymisé
DecouperSplitsUseCase	Répartir en train/val/test stratifié
VerifierRepartitionSplitsUseCase	Vérifier la répartition réelle des splits
ExtraireSousEnsembleSftUseCase	Extraire le sous-ensemble SFT publiable
ExtraireSousEnsembleDpoUseCase	Extraire le sous-ensemble DPO publiable
EvaluerBaselineZeroShotUseCase	Évaluer la baseline zero-shot (référence)
Étape 2 : SFT	
FormaterDatasetChatMLUseCase	Rendre le pivot en ChatML pour l'entraînement
EntraînerSftUseCase	Orchestrer un run d'entraînement SFT-LoRA
AjusterBoucleHyperparametresSftUseCase	Boucle bornée d'ajustement d'hyperparamètres
SauvegarderCheckpointSftUseCase	Persister les métadonnées d'un checkpoint
Étape 3 : DPO	
ReformulerPreferenceDpoUseCase	Reformuler le "chosen" au format cible avant DPO
FormaterDatasetChatMLPreferenceUseCase	Rendre les paires de préférence en triplets
EntraînerDpoUseCase	Orchestrer un run d'entraînement DPO
Étape 4 : Déploiement	
PoursuivreEntretienUseCase	Un tour d'entretien clinique adaptatif
ObtenirDiagnosticUseCase	Déclencher le diagnostic final structuré
ReformulerDiagnosticJsonUseCase	Deuxième tentative de reformulation JSON, à la demande

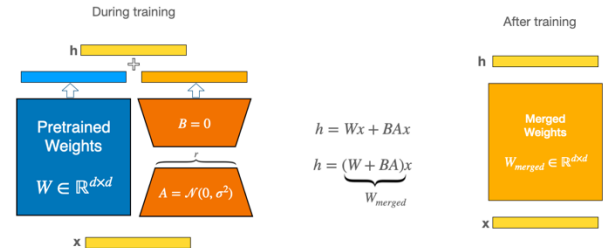
Cette granularité n'est pas accidentelle : chaque cas d'usage porte une seule responsabilité, et chaque adaptateur peut être remplacé sans toucher au domaine métier ni aux autres. C'est exactement ce découplage qui a permis d'ajouter le second essai de reformulation JSON (§4.4) en un seul fichier nouveau, sans modifier aucun des vingt cas d'usage déjà existants.

Concepts clés

Quatre notions techniques structurent ce rapport ; elles sont introduites brièvement ici pour un lecteur non spécialiste, puis reprises en contexte dans les sections correspondantes.

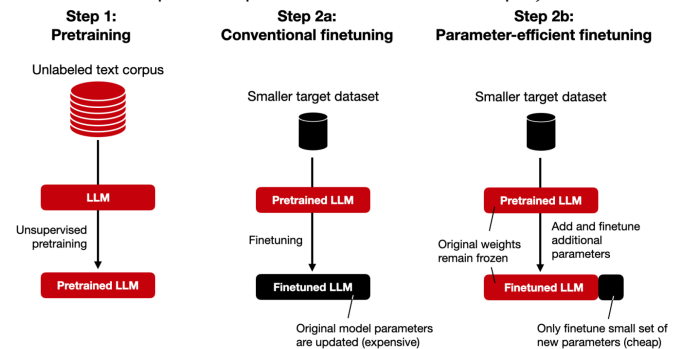
LoRA (Low-Rank Adaptation)

Au lieu de réentraîner les milliards de paramètres d'un modèle de langage, LoRA insère de **petites matrices supplémentaires de rang réduit** dans certaines couches (typiquement l'attention) et n'entraîne que celles-ci. Le modèle de base reste inchangé et gelé ; l'adaptateur LoRA, lui, ne pèse que quelques dizaines de mégaoctets et peut être chargé ou déchargé à la volée.



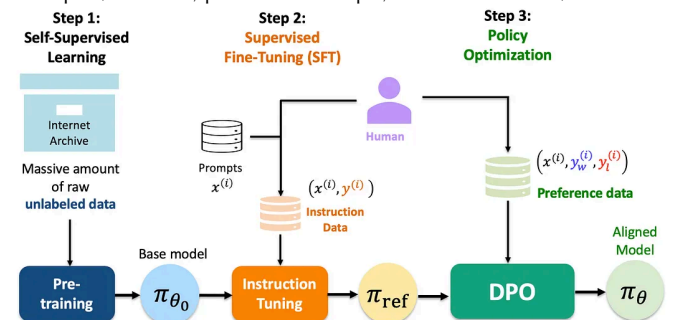
SFT (Supervised Fine-Tuning)

Spécialisation supervisée classique : le modèle apprend, exemple par exemple, à **reproduire une réponse attendue à partir d'un prompt donné**. C'est la première phase d'entraînement de ce projet (§2).



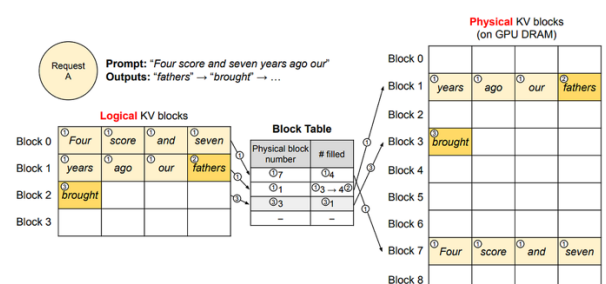
DPO (Direct Preference Optimization)

Seconde phase d'alignement, qui n'apprend plus à partir d'une seule réponse « vraie » mais à partir de **paires (réponse préférée, réponse rejetée)** : le modèle apprend à préférer la première à la seconde, ce qui permet d'orienter des qualités plus difficiles à spécifier par un simple exemple (concision, pertinence clinique, absence de dérive).



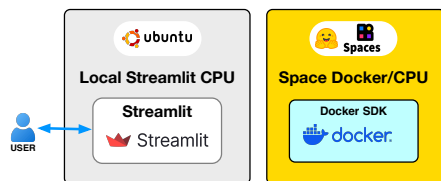
vLLM et PagedAttention

Moteur de service d'inférence imposé par le cahier des charges. **PagedAttention** gère la mémoire GPU du cache d'attention par pages (comme la mémoire virtuelle d'un système d'exploitation) plutôt que par bloc contigu réservé à l'avance, ce qui permet de servir plusieurs requêtes simultanées sans gaspillage de VRAM. **vLLM expose une API compatible OpenAI** et sait **servir un adaptateur LoRA** nativement, sans jamais le fusionner avec le modèle de base.



Environnement de développement

Le projet distingue deux environnements de travail, chacun avec son propre jeu de dépendances Python (gérées par `uv`) :



Un Environnement A Local (WSL2, sans GPU, 5 Go de RAM),	Un Environnement B Distant (GPU à la demande via Hugging Face Jobs et Spaces Dev Mode),
Utilisé pour la préparation des données et le développement courant,	Utilisé pour l'entraînement et l'évaluation coûteuse.

Un **script de vérification** dédié à chacun (`check\_env\_local.py`, `check\_env\_gpu.py`, `check\_env\_remote\_hf.py`) contrôle les prérequis avant de lancer une étape, plutôt que de découvrir un problème de configuration en cours de run facturé.

La **dépendance `torch`** est résolue différemment selon l'environnement actif (mécanisme `tool.uv.sources` de `uv`) : une version CPU légère pour l'Environnement A, la version CUDA complète pour l'Environnement B, un défaut d'origine (`torch` toujours résolu en version CUDA, même en local) a été corrigé pendant ce projet et documenté en section Dépannage du README.

La **suite de tests automatisés** couvre le domaine métier, les adaptateurs (avec des doubles en mémoire pour ceux qui nécessitent normalement un GPU ou un réseau réel) et la couche API. Elle s'exécute entièrement dans l'Environnement A, sans GPU, ce qui la rend gratuite à relancer aussi souvent que nécessaire, c'est cette même suite qui s'exécute dans le pipeline CI/CD (\$4.3).

Cette séparation n'est pas qu'une question d'organisation : elle a un coût réel. Travailler et tester sans GPU permanent réduit la facture du projet à presque rien pour tout le développement quotidien, seules les étapes d'entraînement et d'évaluation elles-mêmes sollicitent une machine louée à l'heure.

Caractéristique	Environnement A	Environnement B
Lieu	Local (WSL2, 5 Go RAM)	Distant (HF Jobs / Spaces Dev Mode)
GPU	Aucun	Oui (typiquement L4)
Extra uv	local	remote
torch	Build CPU	Build CUDA
Dépendances clés	pandas, ydata-profiling, presidio-analyzer/anonymizer, spaCy, transformers, mlflow, httpx	torch CUDA, transformers, trl, peft, accelerate, bitsandbytes, vllm, mlflow
Utilisé pour	Préparation des données (\$1), tests automatisés, développement quotidien	Entraînement SFT/DPO (\$2, \$3), évaluation GPU, déploiement réel (\$4)
Coût	Quasi nul	Facturé à l'heure

Et voici le tableau des clés et secrets pour l'environnement distant B :

Type	Visibilité	Clé	Description / Rôle
Variables	Public	CHSA_JOURNAL_AUDIT	URL, chemin ou identifiant du journal d'audit
Variables	Public	CHSA_JOURNAL_AUDIT_REPO	Dépôt associé au journal d'audit
Secrets	Private	CHSA_CLE_API_DEMO	Clé d'API pour l'environnement de démonstration
Secrets	Private	HF_TOKEN	Jeton d'authentification Hugging Face (pour Jobs / Spaces)

1. Préparation des données

1.1 Méthodologie

Quatre corpus publics ont été collectés et unifiés dans un schéma pivot commun (JSON, bilingue FR/EN) :

- **MediQAI** (questions/réponses médicales, déclinées en trois sous-formats OEQ, MCQU, MCQM),
- **FrenchMedMCQA** (QCM pharmacie/médecine),
- **MedQuAD** (questions/réponses médicales) et
- **UltraMedical-Preference** (paires choisies/rejetées, seule source directement exploitable pour le DPO sans reformulation).



Chaque exemple pivot porte un identifiant déterministe (`chsa-<source>-<uuid>`), une source, une langue, un type (sft ou dpo) et, lorsqu'ils sont disponibles, des champs cliniques structurés (symptômes, antécédents, constantes vitales : pression artérielle, fréquence cardiaque, SpO2, fréquence respiratoire).

La fusion des quatre sources en un pivot unique déduplique par **identifiant déterministe** : sur l'ensemble des exemples collectés, **12 321 doublons exacts ont été détectés** et écartés (archivés séparément plutôt que simplement supprimés, pour traçabilité).

Combiner quatre corpus d'origines différentes pose un problème simple à énoncer mais pas si simple à résoudre : chacun a son propre format, ses propres champs, parfois sa propre langue. Le schéma pivot sert justement à ça, à donner à chaque exemple, quelle que soit sa source d'origine, la même structure de base, pour que tout le reste du pipeline (anonymisation, découpage, entraînement) puisse travailler sans se soucier d'où vient chaque exemple.

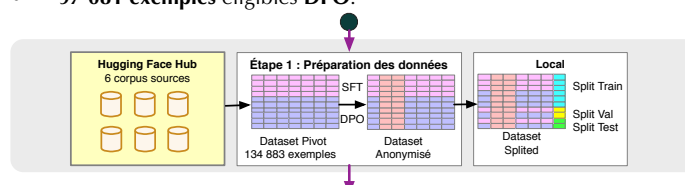
1.2 Anonymisation et contrôle qualité

L'**anonymisation (Microsoft Presidio)**, imposée au cahier des charges) retire les identifiants directs (noms, dates, identifiants de dossier, coordonnées) avant toute publication sur le Hub Hugging Face. Presidio fonctionne par détection d'entités (reconnaisseurs dédiés par **type de PII**, combinables avec des règles spécifiques au domaine médical) suivie d'une stratégie de remplacement configurable ; une révision manuelle de la PII résiduelle et un contrôle qualité systématique sur un échantillon stratifié complètent la détection automatique, qui n'est jamais supposée parfaite à elle seule sur un texte clinique libre.

Concrètement, Presidio reconnaît par exemple les noms propres, les dates, les numéros de téléphone ou les adresses grâce à des règles et des modèles dédiés à chaque type d'information. Mais un texte clinique libre, rédigé par un soignant dans le feu de l'action, échappe parfois à ces règles : une faute de frappe, une tournure inhabituelle, une information sensible qui ne rentre dans aucune catégorie prévue. D'où la double vérification prévue dans ce projet, la détection automatique fait le gros du travail, la révision manuelle sur échantillon rattrape ce qui lui échappe.

L'anonymisation complète a porté sur **134 883 exemples** (pivot dédupliqué). Le découpage final, stratifié par source et vérifié automatiquement (écart maximal toléré entre strates), répartit ces exemples en :

- **37 802 exemples éligibles SFT** et
- **97 081 exemples éligibles DPO**.



1.3 Résultat

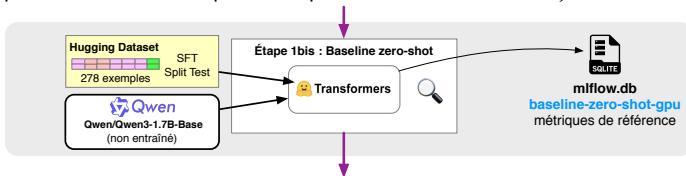
Le cahier des charges (Livrable 1) demandant un jeu d'entraînement de l'ordre de **5000 paires**, un sous-ensemble filtré et stratifié de cette taille est extrait de chaque split (SFT et DPO) pour publication et entraînement réel, le pivot complet (134 883 exemples) reste disponible pour d'éventuels travaux futurs à plus grande échelle. L'ensemble est publié et versionné sur le Hub Hugging Face (dépôts privés dédiés à chaque étape), bilingue, anonymisé et prêt pour l'entraînement.

Étape	Volume
Exemples collectés (4 corpus bruts)	134 883 + 12 321 doublons écartés
Pivot anonymisé (après déduplication)	134 883 exemples
Split éligible SFT	37 802 exemples
Split éligible DPO	97 081 exemples
Sous-ensemble publié SFT (Livrable 1)	5 000 exemples
Sous-ensemble publié DPO (Livrable 1)	5 000 exemples
Jeu de test clinique (évaluation)	278 exemples

Tableau 1, Volumes réels du jeu de données à chaque étape.

1.4 Baseline zero-shot

L'évaluation **baseline zero-shot** (modèle non spécialisé) sert de point de comparaison avant tout entraînement, sur le même jeu de test clinique (278 exemples) que les évaluations post-SFT et post-DPO. Sans cette référence, il serait impossible de dire si un gain de F1 après entraînement vient réellement du fine-tuning, ou s'il aurait été là de toute façon avec le modèle de base seul, mal formulé ou insuffisamment guidé par le prompt. C'est ce même jeu de test, strictement identique, qui sera réutilisé sans aucune modification pour les évaluations post-SFT (§2.3) et post-DPO (§3.3), afin que la comparaison entre les trois soit juste.



Tableau, Résultats de la baseline zero-shot (modèle non entraîné), mêmes 278 exemples pour les deux runs

Modèle / Configuration	Exact match	F1 moyen (token)	Latence moyenne	Échecs d'inférence
CPU (Q4_K_M, quantifié)	0	0,037	~21,6 s	36/278
GPU (bf16, pleine précision)	0	0,043	~7,3 s	0/278

La baseline GPU est plus rapide, plus fiable (zéro échec) et légèrement meilleure en F1 : ces deux points zéro servent de référence mesurable pour juger l'effet du SFT (§2.3).

2. SFT + LoRA

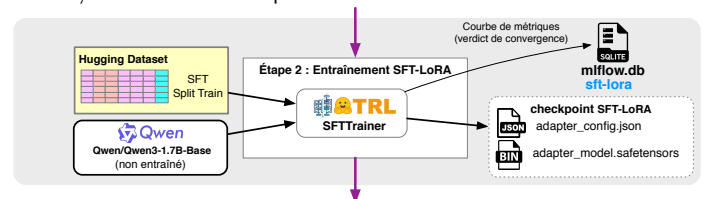
2.1 Méthodologie

Le modèle de base Qwen3-1.7B-Base est spécialisé par fine-tuning supervisé (SFT) avec un adaptateur LoRA plutôt qu'un fine-tuning complet : seule une fraction des paramètres est entraînée, ce qui réduit drastiquement le coût mémoire et de calcul tout en conservant les capacités générales du modèle de base, un compromis adapté à un POC avec budget de calcul contraint. Le chargement s'effectue en quantification 4-bit ('BitsAndBytesConfig') pour l'entraînement sur une seule GPU distante (HF Jobs).

Paramètre	Valeur	Rôle
Quantification	nf4, double, bf16	Divise par 4 la VRAM du modèle base
Rang LoRA (r)	16	Capacité de l'adaptateur
Alpha LoRA	32 (2r)	Échelle de la mise à jour, règle standard
Dropout LoRA	0,05	Régularisation anti-surapprentissage
Modules cibles	q, k, v, o_proj	Attention seulement, adaptateur léger
Taux d'apprentissage	$2 \cdot 10^{-4}$	Valeur typique LoRA
Époques	3	Compromis apprentissage/mémorisation
Taille de lot	4	Limité par la VRAM disponible
Packing	activé	GPU utilisé à ~100% (vs 40-60% en padding classique)

Tableau 2, Configuration réelle de l'entraînement SFT-LoRA (recipes/sft_qwen3_lora.yaml, README « Explication du contenu de la recette »).

Le type de **quantification 'nf4'** (« NormalFloat 4-bit ») est un format numérique optimisé pour des poids à distribution approximativement gaussienne, plus précis à taille égale que le format 'fp4' générique, à ne pas confondre avec le garde-fou de sécurité clinique NF4 discuté en §5, homonyme sans lien technique.



2.2 Résultat

Le premier entraînement réel a été mené à son terme sur GPU L4 en environ 20 minutes (342 pas, 3 époques), verdict de **convergence SAINE**.

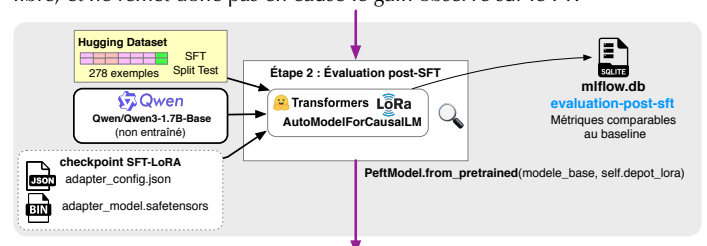
Le **checkpoint SFT-LoRA** atteint un **F1 (token) d'environ 0,112** sur le jeu de test clinique, la valeur de référence à laquelle le DPO est ensuite comparé (§3). C'est ce checkpoint qui sert de politique de référence (π_{ref}) pour l'alignement par préférences, et ses poids sont publiés durablement sur le Hub (dépôt privé dédié).

2.3 Evaluation Post-SFT

Après l'entraînement SFT-LoRA décrit en §2.1, le modèle réellement entraîné (poids de base + adaptateur LoRA) a été évalué sur le même sous-ensemble de 278 exemples et selon les mêmes métriques que les baselines de la section 2.2 (Exact Match, F1 moyen au niveau token, latence moyenne), afin de permettre une comparaison directe et non biaisée entre les trois configurations. Le résultat est net :

Le F1 moyen passe de 0,043 (meilleure baseline, GPU bf16) à 0,112 après SFT, soit un score quasiment triplé, première preuve chiffrée d'un effet mesurable du fine-tuning, cohérente avec le verdict de convergence saine obtenu en §2.2.

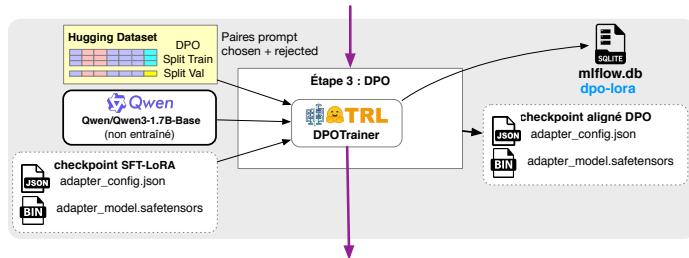
L'**Exact Match**, en revanche, reste à 0,000 sur les trois runs (CPU, GPU, post-SFT) ; ce résultat est attendu, la métrique exigeant une correspondance caractère-à-caractère avec des réponses de référence rédigées en langage libre, et ne remet donc pas en cause le gain observé sur le F1.



3. DPO (Direct Preference Optimization)

3.1 Méthodologie

Le DPO aligne le modèle sur **des paires de préférence (réponse choisie / réponse rejetée)** sans modèle de récompense séparé : la perte compare directement, sous forme de log-vraisemblance, la politique en cours d'entraînement à la politique de référence figée (le checkpoint SFT). L'**hyperparamètre β** contrôle la force de cet ancrage à la référence : plus β est faible, plus le modèle est autorisé à s'écarter de π_{ref} , au risque d'une dérive de génération non capturée par la seule perte d'entraînement (§3.2). En pratique, cela revient à dire au modèle « je préfère quand tu réponds comme ceci plutôt que comme cela », sans jamais lui donner une réponse parfaite à copier, seulement une préférence entre deux réponses possibles. Le réglage de β s'est révélé être le point le plus délicat de cette étape : réglé trop bas, le modèle s'autorise à trop s'éloigner de ce qu'il savait déjà bien faire, avec le risque concret observé dans ce projet, une dégénérescence complète de la génération (§3.2 et §3.3).



3.2 Métriques et itérations réelles

La mise au point de β a nécessité plusieurs itérations réelles, chacune diagnostiquée sur des métriques concrètes (rewards/accuracies, rewards/margins) plutôt que sur la seule perte d'entraînement brute :

Run	β	Taille	Verdict	rewards/accuracies	rewards/margins
1 (batch=1)	0,1	100	sous-apprentissage	0,30	-1,25
2 (lr relevé)	0,1	100	sous-apprentissage*	0,625 / 0,75	+0,72 / +0,60
3	0,1	5000	saine	0,725	+3,00
4	0,3	100	sous-apprentissage*	0,60	1,93
5 (final)	0,3	5000	saine	0,75	4,84

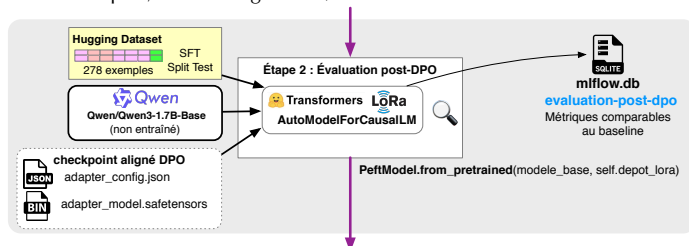
Tableau 3, Journal des runs DPO (extrait, README §3.2). *Verdict basé sur un seuil de perte hérité du SFT, non recalibré pour DPO ; rewards/accuracies et margins indiquent un apprentissage réel malgré ce verdict.

Le **run à $\beta=0,1$ sur 5000 exemples** atteint un rewards/margins anormalement élevé (+3,00) : signe d'un ancrage trop faible à π_{ref} .

3.3 Évaluation post-DPO

L'évaluation post-DPO correspondante révèle une **dégénérescence de génération réelle (changements de langue aléatoires FR/EN vers JA/ZH/AR, effondrements en répétitions)** non visible dans la perte d'entraînement seule.

Relever β à 0,3 corrige ce défaut, confirmé sur les deux échelles (100 puis 5000 exemples, aucune régression).



β (échelle)	repetition_penalty	F1 (token)	Latence
0,1	—	0,049	~12,0 s
0,1	1,2	0,063	~11,9 s
0,3 (100)	1,2	0,112	~10,7 s
0,3 (5000, final)	1,2	0,110	~11,2 s

Tableau 4, Évaluation post-DPO (checkpoint mombastic/chsa-triage-dpo-lora, README §3.3). $\beta=0,3$ + repetition_penalty=1,2 ramène le F1 au niveau du post-SFT (0,112), sans régression au passage à l'échelle (100 → 5000 exemples).

Hyperparamètres finaux retenus :

- $\beta=0,3$,
- **taux d'apprentissage** $5 \cdot 10^{-5}$,
- **taille de lot** 4,
- **rang LoRA** 16 (recipes/dpo_qwen3_lora.yaml).

Le **checkpoint DPO** n'est jamais fusionné avec la base : il est servi nativement par vLLM comme adaptateur LoRA (§4), ce qui permet de revenir au modèle de base à tout moment sans réentraînement, et de comparer facilement plusieurs versions du garde-fou d'alignement sans dupliquer les 3,2 Go de poids du modèle de base.

3.4 Portée de la reformulation des préférences

Les paires de préférence brutes d'UltraMedical-Préférence ne portent pas nativement le format de sortie structuré ciblé par ce projet (niveau ESI, catégorie, ressources) : un cas d'usage dédié reformule chaque paire choisie/rejetée dans ce format avant l'entraînement DPO proprement dit.

Ce découplage reformulation/DPO, les réponses choisies ne sont pas reformulées, seul le prompt système change, reste une cause probable, distincte de la dégénérescence de fluidité (§3.2), de la perte partielle du format JSON constatée à l'évaluation : la mission ne demande que l'alignement SFT+DPO sur les préférences, le format JSON strict étant un ajout du cahier des charges local visé plutôt au niveau du prompting de l'endpoint déployé

3.5 Tentative de mitigation à l'inférence

Une deuxième voie, distincte du réentraînement, a été testée pour pallier la non-conformité du format JSON constatée en §3.3 : un second appel au modèle, à la demande explicite de l'infirmier, réutilisant le prompt de reformulation déjà écrit pour la préparation des données DPO (§3.4). Le mécanisme lui-même (nouveau cas d'usage, nouvel endpoint, journal d'audit dédié) a été vérifié de bout en bout sans erreur.

En revanche, testé contre le checkpoint DPO réel, aussi bien en local (CPU, sans GPU) qu'une fois sur le Space GPU réel, cette seconde tentative n'a pas résolu le problème de fond : le modèle a montré trois défauts de fiabilité distincts, indépendants du mécanisme lui-même.

Tableau, Trois défauts réels observés lors des tests d'inférence

Défaut	Observation réelle
Répétition	Réponses quasi identiques répétées en boucle d'un tour à l'autre
Digression	Questions et remarques sans rapport avec le cas du patient (état civil, chimiothérapie)
Blocage en raisonnement	Sous contrainte de schéma JSON strict, le modèle reste bloqué en phase de raisonnement libre sans jamais atteindre la sortie structurée

Ce constat confirme que le format JSON forcé au niveau du serveur d'inférence (guided_json, déjà recommandé en §7.1) garantit une syntaxe valide une fois cette phase atteinte, mais ne garantit pas que le modèle l'atteigne. La fiabilité du checkpoint actuel, dans ces conditions, reste le facteur limitant, plus que le mécanisme de reformulation ou de contrainte de format en eux-mêmes.

4. Déploiement

Jusqu'ici, tout se jouait hors ligne : des jeux de données, des checkpoints, des métriques calculées après coup. Le déploiement change la nature de l'exercice, il s'agit de faire tourner ce même modèle en continu, de répondre à de vraies requêtes dans un délai raisonnable, et de le faire de façon suffisamment robuste pour rester disponible sans surveillance constante. C'est aussi l'étape où des défauts invisibles à l'évaluation hors ligne peuvent apparaître, comme le montre l'incident détaillé en §4.2.

4.1 Architecture retenue

Décision produit prise pendant le déploiement : une architecture à deux pièces plutôt que trois.

Un unique **Space Hugging Face Docker/GPU** héberge, dans le même conteneur, l'**API FastAPI** et le **serveur vLLM** (un seul coût GPU, un seul on/off) ;

Le **frontend Streamlit** de test tourne en **local** sur le poste de l'opérateur, car il ne fait qu'appeler l'API en HTTP sans aucun calcul GPU, un Space dédié uniquement pour cette interface aurait exigé un abonnement payant sans bénéfice réel.

Le frontend ne sait jamais à l'avance si le modèle est chargé : il sonde en boucle un endpoint de santé applicatif qui répond toujours HTTP 200, y compris pendant les plusieurs minutes de chargement du modèle, avec un corps structuré distinguant explicitement « serveur d'inférence indisponible » d'une véritable erreur, un simple health-check Docker qui redémarrerait le conteneur sur ce délai normal aurait été contre-productif.

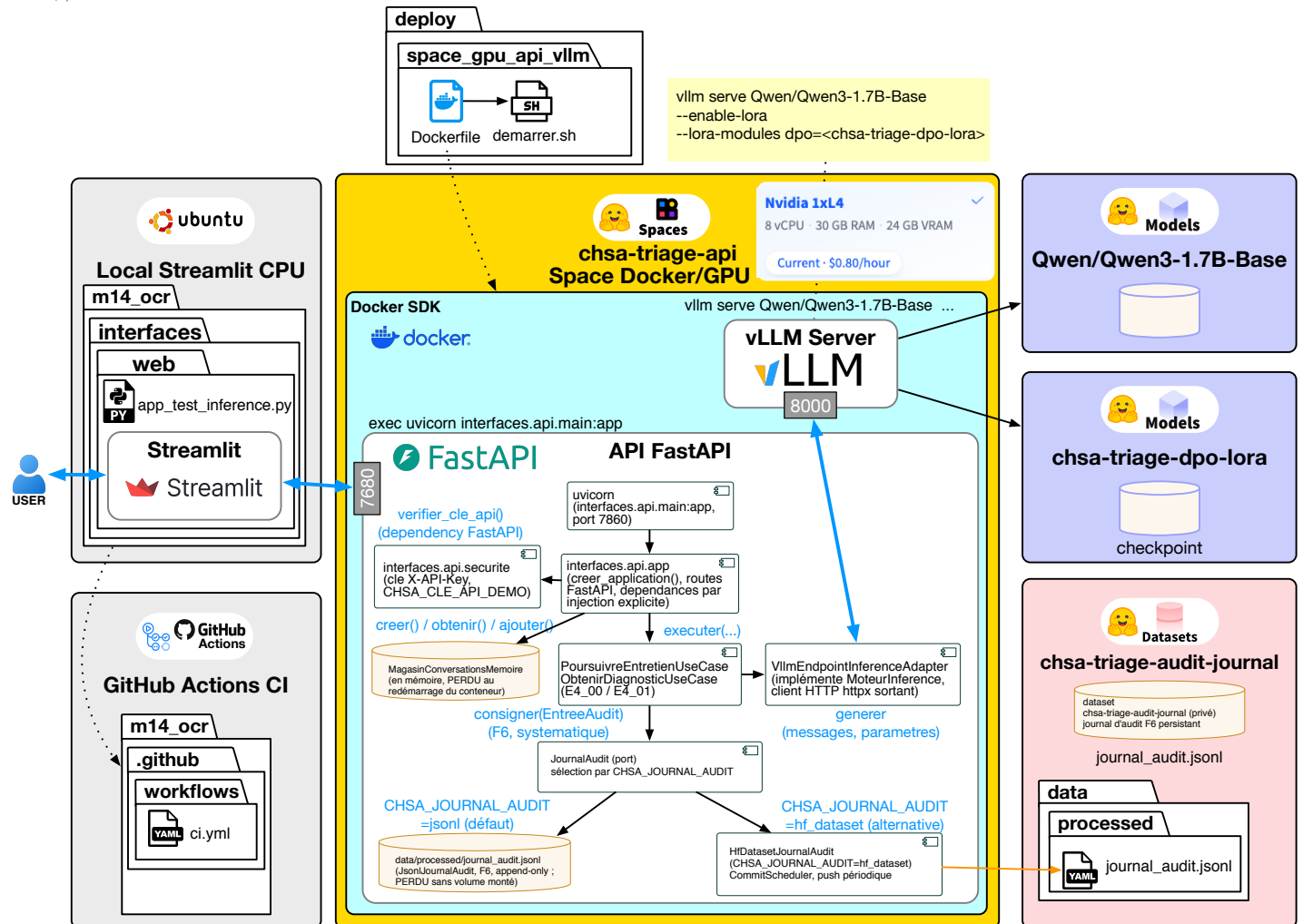


Figure 9, Architecture de déploiement réelle : Space Docker/GPU (API+vLLM), frontend local, journal d'audit persistant.

Dans le conteneur unique, deux processus tournent côte à côte :

- **vllm serve** en arrière-plan, et
- **l'API FastAPI** au premier plan.

Le script de démarrage ne bloque jamais sur le chargement du modèle, l'API répond immédiatement, même pendant que vLLM charge encore ses poids en arrière-plan. C'est précisément pour cette raison que /sante doit sonder l'état réel de vLLM plutôt que supposer sa disponibilité dès le démarrage du conteneur.

Le serveur vLLM est lancé avec `--enable-lora --lora-modules dpo=mombastic/chsa-triage-dpo-lora --max-lora-rank 16`, sur le modèle de base Qwen/Qwen3-1.7B-Base.

L'adaptateur DPO n'est jamais fusionné avec la base : vLLM le charge à côté et l'applique à la volée, en le désignant par le nom `dpo` dans chaque requête. Concrètement, cela permettrait de servir plusieurs adaptateurs en parallèle (SFT, DPO, une future version) sans jamais dupliquer les 3,4 Go de poids du modèle de base, un simple changement de nom suffirait à basculer de l'un à l'autre.

Le trajet complet d'une requête suit ce chemin :

- le frontend Streamlit appelle l'API FastAPI en HTTP (port 7680, exposé),
- qui appelle à son tour vLLM en interne (port 8000, jamais exposé hors du conteneur) via `VllmEndpointInferenceAdapter`.
- La réponse remonte la même chaîne en sens inverse, avec une entrée consignée dans le journal d'audit à chaque étape (F6).

4.2 Incident technique majeur et résolution

Le premier déploiement réel s'est heurté à un **segfault natif** (crash silencieux du processus, sans message Python exploitable) systématique de vLLM, survenant à chaque fois exactement au même point : juste après le chargement des poids du modèle, pendant le profilage interne que vLLM effectue pour dimensionner son cache de mémoire GPU, sur l'instance GPU réelle (NVIDIA L4).

Huit pistes ciblées ont été testées méthodiquement en conditions réelles, chacune produisant le même échec identique malgré le changement, la liste complète, dans l'ordre où elles ont été écartées :

#	Piste testée	Résultat
1	Désactivation du multiprocessing interne de vLLM (variable d'environnement dédiée)	Sans effet, cette variable ne s'applique qu'à l'usage programmatique de vLLM, jamais au serveur lancé en ligne de commande
2	Exécution du conteneur sous un utilisateur non root explicite (au lieu de root par défaut)	Correction légitime conservée, mais sans effet sur le crash
3	Adaptateur LoRA désactivé entièrement	Même crash identique, sans le module de chargement LoRA en cause
4	Moteur d'attention alternatif (FlashInfer au lieu du moteur par défaut)	Même crash identique
5	Ordonnancement asynchrone désactivé	Même crash identique
6	Délai fixe de démarrage avant le lancement de vLLM	Même crash identique, écarte un GPU/pilote pas encore stabilisé
7	Version de vLLM différente (deux versions testées)	Même crash identique sur les deux versions
8	Désaccord pilote NVIDIA / version CUDA (vérifié directement via l'outil de diagnostic du pilote)	Pilote et CUDA confirmés sains, écarté

Tableau 5. Les huit pistes de diagnostic testées et écartées pour le segfault de démarrage vLLM (issue publique vllm-project/vllm#58616).

Un test décisif a permis de caractériser la nature du problème sans le résoudre directement : **activer une trace complète de chaque appel de fonction Python** (diagnostic officiel de vLLM, qui ralentit l'exécution de plus de 100 fois) a permis, à lui seul, un démarrage complet et sain. Ce ralentissement massif faisant disparaître le crash est la signature caractéristique d'une condition de course dépendante du temps réel d'exécution, un défaut jamais isolé à une seule cause identifiable dans le code du projet ou de vLLM lui-même.

La résolution effective est venue d'un changement de stratégie plutôt que d'un neuvième correctif ciblé : reconstruire l'image du Space à partir de l'**image Docker officielle vllm/vllm-openai** (construite et testée par l'équipe vLLM elle-même, avec son propre environnement Python cohérent) plutôt que d'installer vLLM manuellement dans une image CUDA générique. Le problème a disparu entièrement, sans qu'une cause racine précise n'ait pu être établie côté vLLM. Cet incident, sa reproduction complète et sa résolution ont été signalés en amont au projet vLLM (issue publique vllm-project/vllm#58616) pour bénéficier à d'autres équipes confrontées au même symptôme.

Deux bugs applicatifs supplémentaires ont ensuite été trouvés et corrigés lors de la toute première conversation réelle contre ce vLLM enfin fonctionnel, un rappel concret que la disponibilité du serveur d'inférence n'implique pas à elle seule la correction du comportement applicatif :

1. Le paramètre de **nombre maximal de tokens générés** utilisait en interne le vocabulaire du moteur local llama.cpp (`n_predict`), jamais traduit vers le nom réellement attendu par l'API compatible OpenAI de vLLM (`max_tokens`), un adaptateur voisin du projet effectuait déjà cette traduction pour un troisième moteur, mais elle avait été omise pour l'adaptateur vLLM, resté non testé en conditions réelles jusqu'à la résolution du segfault. Le champ inconnu était ignoré silencieusement par vLLM, laissant la génération partir jusqu'à la limite par défaut du modèle (2048 tokens) sans aucune limite de tour de conversation.
2. Aucune **température de génération** n'était fixée explicitement dans les cas d'usage d'entretien et de diagnostic, laissant vLLM retomber sur un échantillonnage aléatoire non validé, alors que l'évaluation post-DPO qui avait validé la pénalité de répétition (§3.3) utilisait systématiquement une génération déterministe. Sans température explicite, le symptôme observé (réponses incohérentes, mélange spontané de langues au sein d'une même réponse) s'est reproduit à l'identique sur deux générations de GPU différentes (L4 en bfloat16,

T4 en float16 par repli automatique), ce qui a permis d'écarter une cause matérielle avant de trouver la cause réelle. Les deux corrections ont rétabli des réponses cohérentes en conditions réelles, vérifiées par une conversation complète de bout en bout.

4.3 CI/CD et endpoint réel

Le **pipeline GitHub Actions** (imposé au cahier des charges) comprend deux jobs : l'exécution de la suite de tests automatisés (448 tests, 0 échec, aucune dépendance GPU) et la vérification que l'image Docker de l'API se construit correctement, à chaque push sur la branche principale. L'endpoint réel (mombastic/chsa-triage-api) répond de façon opérationnelle : chargement du modèle et de l'adaptateur LoRA DPO, healthcheck applicatif ne renvoyant jamais d'erreur brute, latence de réponse observée de l'ordre de 5 à 30 secondes par tour d'entretien selon le matériel GPU alloué (L4 ou T4, en fonction de la disponibilité côté fournisseur), cette latence n'a pas encore fait l'objet d'une mesure statistique systématique sur un échantillon représentatif (recommandation \$6.1).

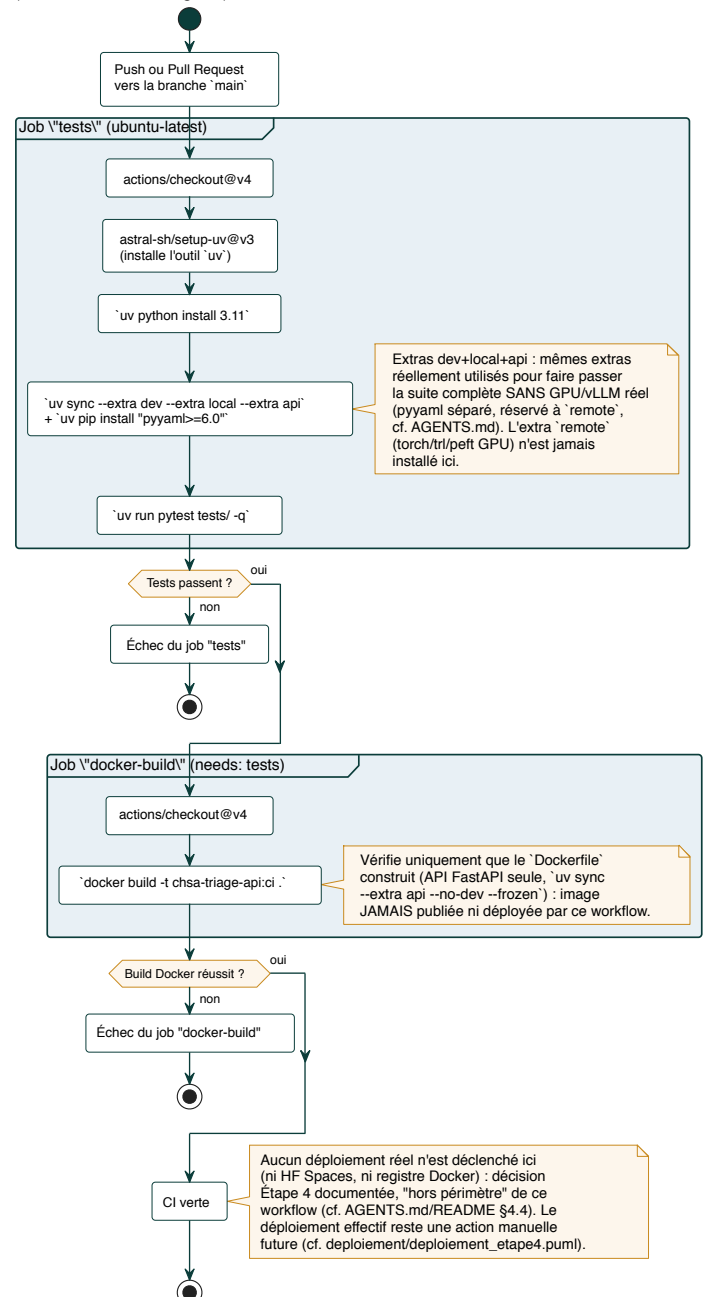


Figure 11. Pipeline CI/CD : tests automatisés + vérification du build Docker.

4.4 Traçabilité et sécurité applicative

Chaque tour d'entretien et chaque demande de diagnostic est consigné dans un **journal d'audit horodaté** (conversation, entrée, sortie, version du modèle), garde-fou de traçabilité imposé au projet, persisté gratuitement dans un dataset privé Hugging Face via un **mécanisme de publication périodique en arrière-plan**, déjà utilisé ailleurs dans le projet pour le suivi d'expérimentation d'entraînement.

Ce choix évite un abonnement de stockage persistant payant tout en garantissant que le journal survit aux redémarrages du conteneur, contrairement à une simple écriture sur le disque éphémère du Space, qui aurait perdu tout l'historique à chaque reconstruction de l'image (redémarrages nombreux pendant l'incident décrit en §4.2).

```
{
  "horodatage": "2026-09-29T07:51:08.367512+00:00",
  "type_evenement": "tour_entretien",
  "conversation_id": "789caef2-23c2-4665-9a63-c535bf969167",
  "entree": "...",
  "sortie": "...",
  "version_modele": "mombastic/chsa-triage-dpo-lora",
  "metadonnees": {}
}
```

Structure de l'historique d'entretien dans le journal **chsa-triage-audit-journal**.

L'accès à l'API est protégé par une clé (en-tête `X-API-Key`), vérifiée par une dépendance FastAPI sur chaque route sensible, distincte du jeton d'accès Hugging Face lui-même utilisé pour les opérations internes (téléchargement de modèle, publication du journal d'audit). Le stockage du jeton Hugging Face a lui-même donné lieu à un incident réel pendant ce déploiement (formulaire web du Space rejetant un caractère du jeton, contournable en le déposant directement via l'API programmatique plutôt que le formulaire), documenté en détail dans le README du projet.

5. Sécurité, conformité et vie privée

Le projet distingue explicitement deux notions homonymes qui portent toutes deux le sigle « NF4 » dans la documentation du projet, à ne pas confondre :

- le **format de quantification nf4** (NormalFloat 4-bit, §2.1, un détail d'implémentation de l'entraînement, déjà en place) et
- le **garde-fou de sécurité clinique NF4** du cahier des charges (un juge automatique censé rejeter toute réponse dont le score de sécurité serait inférieur à 4 sur 7, non encore conçu ni implémenté, voir statut réel en §6).

La protection des données personnelles s'appuie sur trois mécanismes complémentaires, à des étapes différentes du cycle de vie de la donnée :

- l'**anonymisation Presidio** en amont de tout entraînement (§1.2, retire les identifiants directs avant publication) ;
- le **contrôle d'accès applicatif par clé d'API** sur l'endpoint déployé (§4.4) ; et
- le **journal d'audit**, qui ne consigne que les échanges cliniques de l'entretien (jamais d'identité de patient, celle-ci restant hors du périmètre technique de l'agent, dont le rôle se limite à l'entretien symptomatique).

Le magasin de **conversations actif** (état d'une conversation en cours) est délibérément gardé hors de l'architecture hexagonale ports/adaptateurs : c'est un état de session éphémère du processus, en mémoire, jamais persisté ni consigné dans le journal d'audit lui-même, seuls les tours d'entretien et les diagnostics le sont. Cette conception a une conséquence directe pour le passage à l'échelle, discutée en §6.3 : elle est incompatible avec plusieurs instances de l'API sans état partagé externe.

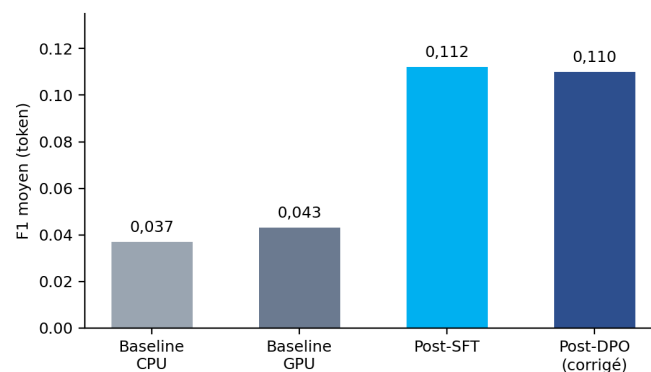
Il faut noter une limite réelle de ce dispositif : l'**anonymisation Presidio ne s'applique qu'aux corpus d'entraînement** (§1.2), jamais aux échanges d'une conversation en cours. Le texte saisi par l'infirmier est transmis tel quel au modèle et consigné tel quel dans le journal d'audit, sans filtrage d'identifiants. Cette limite est cohérente avec le périmètre du POC (l'agent ne recueille explicitement aucune identité de patient, §5), mais mérite d'être tranchée avant tout déploiement clinique réel, si le texte libre saisi venait à contenir malgré tout une information identifiante.

6. Analyse des résultats

Statut réel de chaque critère d'acceptation du POC (cahier des charges §9), à date :

Critère	Statut
Endpoint vLLM opérationnel, latence documentée	Atteint (§4.3)
Pipeline CI/CD exécute les tests à chaque push	Atteint (448 tests, 0 échec)
JSON de diagnostic valide sur $\geq 95\%$ des requêtes	Non atteint, format non toujours respecté en sortie réelle de vLLM (voir §7)
Accuracy ESI > baseline zéro-shot, mesurable	Non mesurable tant que le format JSON n'est pas fiable
Aucune réponse < 4/7 en sécurité (garde-fou NF4)	Non implémenté, juge de sécurité clinique non encore conçu

Tableau , Statut réel des critères d'acceptation du POC (cahier des charges §9).



Figure, Évolution du F1 moyen (token) à travers les quatre étapes d'évaluation : baseline CPU, baseline GPU, post-SFT, post-DPO corrigé ($\beta=0,3$, README §3.3).

Le F1 (0,110–0,112) reste modeste en valeur absolue, mais l'analyse comparative montre que la variable décisive n'était pas la seule qualité de l'alignement DPO : un paramétrage d'inférence incomplet (§4.2) suffisait à lui seul à produire des réponses inexploitable, indépendamment de la qualité réelle du modèle.

Ce constat, qu'un modèle correctement aligné peut sembler défaillant à cause d'un maillon d'infrastructure non validé, est probablement le résultat méthodologique le plus transférable de ce POC pour la suite du projet : **évaluer un modèle uniquement hors ligne, sans jamais l'exercer contre le serveur d'inférence réellement visé pour la production, laisse une classe entière de défauts invisible jusqu'au déploiement.**

Concrètement, un F1 de 0,11 ne veut pas dire que le modèle se trompe neuf fois sur dix : la métrique mesure un chevauchement de mots au niveau du token entre la réponse générée et une réponse de référence rédigée en langage libre, pas un jugement binaire correct/incorrect. Deux réponses cliniquement équivalentes mais formulées différemment obtiennent mécaniquement un score imparfait. C'est pour cette raison que l'accuracy de classification ESI, une fois le format JSON strict en place (§7.1), sera une mesure bien plus parlante que le F1 seul pour juger de la qualité clinique réelle de l'agent.

Sur le plan de l'ingénierie du projet, deux disciplines se sont révélées particulièrement rentables pendant cette mission :

- **ne jamais accepter une affirmation sans preuve directement vérifiable** (logs réels, code source lu plutôt que supposé, tests exécutés plutôt qu'estimés), et
- **traiter chaque correctif de déploiement comme une hypothèse à valider explicitement plutôt qu'une certitude**, la moitié des huit pistes du Tableau 5, par exemple, semblaient plausibles a priori et ont dû être écartées une à une par des tests réels, pas par intuition.

7. Roadmap et recommandations pour le passage à l'échelle

Les recommandations suivantes sont organisées par horizon, et découlent directement des écarts réels constatés pendant cette mission (§6) plutôt que de bonnes pratiques génériques de mise en production de LLM.

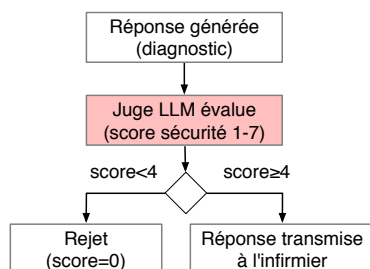
7.1 Court terme (avant toute extension de périmètre)

Contraindre la **sortie du diagnostic à un schéma JSON strict** au niveau du serveur d'inférence (grammaire de génération contrainte, disponible nativement côté vLLM et déjà exposée par sa configuration de sorties structurées) plutôt que par le seul prompt, c'est l'écart le plus bloquant identifié (§6) et le plus directement actionnable : il conditionne à la fois le critère d'acceptation JSON et la mesure d'accuracy ESI, aujourd'hui non mesurable pour cette seule raison.

```
<lim_start>user
Symptômes : « Dyspnée aiguë, cyanose, FR>30/min, SpO2 88 % »<lim_end>
<lim_start>assistant
```

```
{
  "niveau": "ESI-1",
  "categorie": "détresse respiratoire aiguë",
  "ressources_estimees": "déchocage / réanimation immédiate"
}
```

Concevoir et implémenter le **juge de sécurité clinique** (garde-fou NF4), aujourd'hui explicitement en attente d'une décision produit : quel modèle ou quelle règle sert de juge, sur quel échantillon il est validé lui-même, et comment son propre taux d'erreur est mesuré avant d'en faire un filtre de sécurité sur lequel s'appuyer.



Le mécanisme envisagé suit un patron standard dit « **LLM-as-judge** » : une fois le diagnostic généré, un second appel (au même modèle ou à un modèle distinct, décision encore ouverte) évalue la réponse selon une grille de sécurité clinique (1 à 7), sans jamais modifier la réponse elle-même. Si le score rendu est inférieur à 4, le score global de la réponse est forcé à 0 : elle est rejetée avant d'atteindre l'infirmier, quelle que soit par ailleurs la qualité clinique de son contenu.

Ce patron réutilise la même architecture déjà en place (port MoteurInference, format de sortie structuré) : le juge n'est techniquement qu'un troisième prompt spécialisé, au même titre que la reformulation JSON (§4.2) ou le diagnostic lui-même.

Trois décisions produit restent ouvertes avant toute implémentation : quel modèle sert de juge (le même checkpoint DPO, un modèle de base plus grand, ou un service tiers spécialisé en modération clinique), sur quel échantillon annoté par des professionnels de santé ce juge est lui-même validé, et quel taux d'erreur (faux négatifs en particulier) est jugé acceptable avant de s'appuyer dessus comme filtre de sécurité.

Mesurer la latence de façon statistique (percentiles p50/p95 sur un échantillon représentatif de conversations réelles, par type de matériel GPU alloué), pas seulement de façon ponctuelle comme actuellement, le journal d'audit déjà persisté (§4.4) porte l'horodatage nécessaire pour cette mesure sans instrumentation supplémentaire.

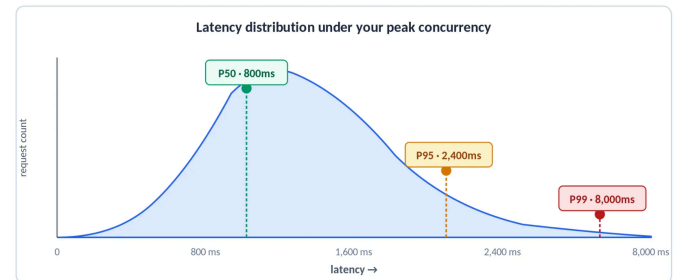
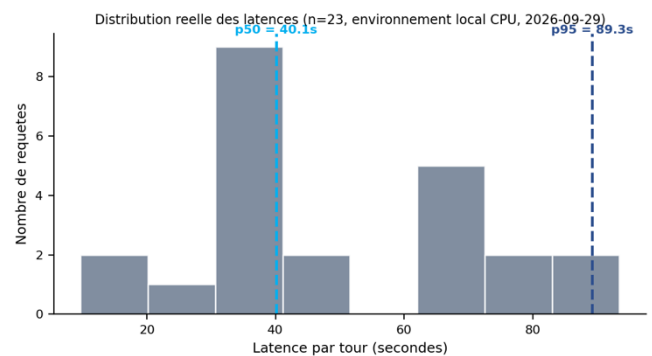


Figure. Courbe générique de latence pour une évaluation de trafic réel
P50 semble correct **Le P95** est le SLA (Service Level Agreement) **Le P99**, c'est l'utilisateur en colère



Note : ces 23 mesures proviennent du **llama-server local (CPU, 2 threads)**, pas du GPU de production. Elles ne doivent en aucun cas être confondues avec la latence réelle observée en production (§4.3, 5 à 30 secondes par tour sur le Space GPU réel), un ordre de grandeur nettement plus favorable que cette mesure locale sans accélération matérielle.

La distribution mesurée n'est pas unimodale : elle forme deux groupes distincts.

- Le premier, concentré autour de 30 à 45 secondes, correspond aux **tours d'entretien** (PoursuivreEntretienUseCase), limités à 128 tokens générés.
- Le second, au-delà de 65 secondes, correspond aux **appels de diagnostic et de reformulation JSON** (ObtenirDiagnosticUseCase, ReformulerDiagnosticJsonUseCase), dont le budget de génération est nettement plus large (respectivement 512 et 256 tokens), pour laisser au modèle la place de produire le raisonnement puis le JSON structuré.

Cette différence de nature entre les deux types d'appels, une question courte contre une sortie structurée complète, explique directement la forme bimodale observée, plutôt qu'une variabilité aléatoire du serveur d'inférence.

7.2 Moyen terme (fiabilisation avant extension d'usage)

Ajouter une supervision applicative minimale : alertes sur les réponses hors format JSON ou en échec, tableau de bord de latence réelle construit directement à partir du journal d'audit déjà persisté (aucune nouvelle télémétrie à mettre en place, seulement à exploiter ce qui existe déjà).

Étendre le jeu de test clinique (278 exemples aujourd'hui) au-delà du périmètre actuel pour fiabiliser statistiquement la mesure d'accuracy ESI une fois le format JSON corrigé.

Documenter et tester une procédure de restauration du service en cas d'indisponibilité prolongée de l'infrastructure GPU sous-jacente, un point de fragilité concret rencontré à plusieurs reprises pendant ce déploiement (échecs d'initialisation de conteneur imputables à la plateforme d'hébergement elle-même, résolu par un **changement de configuration matérielle** plutôt que par une action côté projet), donc indépendant du code du projet mais bien réel en exploitation.

Ajouter une interface de reprise explicite (« nouveau patient ») dans le frontend de test, déjà identifiée et corrigée pendant ce projet, sans elle, un opérateur poursuivant la saisie après un diagnostic mélangeait silencieusement le patient suivant avec l'historique du précédent, un risque d'erreur clinique par confusion d'identité d'entretien qui mérite une vigilance équivalente dans toute interface future (Réalié).



Figure : Bouton "nouveau patient" ajouté lors des dernières mises à jour.

7.3 Long terme (déploiement à l'échelle du CHSA)

Avant toute mise en production clinique réelle : **validation médicale formelle des diagnostics proposés par un comité clinique du CHSA sur un échantillon représentatif**, étude d'impact réglementaire (le logiciel relève potentiellement du cadre des dispositifs médicaux logiciels selon l'usage final retenu), et mécanisme de retour d'expérience structuré depuis le terrain vers une boucle de réentraînement périodique du modèle plutôt qu'un modèle figé une fois pour toutes.

Sur le plan technique, **prévoir une architecture à plusieurs instances de l'API avant toute charge réelle** : le magasin de conversations actuel est volontairement en mémoire (§5), donc incompatible avec plusieurs instances sans état partagé externe (base de données ou cache partagé).

Dimensionner le budget GPU sur la charge réelle observée en **usage clinique** plutôt que sur l'usage ponctuel de ce POC, le choix du matériel (L4 ou équivalent) devra être revalidé une fois un volume de requêtes concurrentes réaliste connu, et non plus supposé.

Enfin, conserver la **discipline de traçabilité** déjà en place (garde-fou F6) comme fondation de toute extension future plutôt que comme une case à cocher : c'est ce même journal d'audit qui permettra, à l'échelle, de mesurer objectivement l'impact clinique réel de l'agent plutôt que ses seules métriques d'entraînement hors ligne.

Conclusion

Ce POC démontre la faisabilité technique d'un **agent de triage médical spécialisé par SFT+DPO** et servi par un **endpoint vLLM réel**, avec une **architecture hexagonale** qui a concrètement facilité le développement et le déploiement (bascule entre trois moteurs d'inférence, local, distant, double de test, sans changement du domaine métier). Chaque étape du pipeline (données, SFT, DPO, déploiement) a produit des **artefacts réels et vérifiables** : jeux de données publiés et versionnés, checkpoints LoRA hébergés sur le Hub, endpoint opérationnel avec pipeline CI/CD vert, journal d'audit persistant.

Le principal apprentissage de la phase de déploiement est que **la validation d'un modèle aligné ne peut pas se limiter à son évaluation hors ligne** : deux défauts d'intégration invisibles en évaluation (traduction de paramètres vers le vocabulaire réel du serveur d'inférence, température de génération jamais fixée explicitement) ont suffi à masquer un alignement DPO par ailleurs correctement validé et mesuré.

Un **incident d'infrastructure majeur** (segfault systématique de vLLM, huit hypothèses testées et écartées avant résolution) a par ailleurs occupé une part significative du temps de déploiement, un rappel que l'intégration d'un composant d'inférence tiers récent, même imposé par le cahier des charges pour de bonnes raisons de performance, reste un risque projet à budgéter explicitement, pas un détail d'exécution.

Les prochaines étapes (§7) sont concrètes, priorisées, et directement issues des écarts réels constatés pendant cette mission plutôt que de recommandations génériques : contraindre le format de sortie, concevoir le garde-fou de sécurité clinique encore manquant, et mesurer statistiquement ce qui n'a pour l'instant été observé que ponctuellement. **Le POC pose une base technique solide** ; le chemin vers un déploiement clinique réel au CHSA reste conditionné à une **validation médicale et réglementaire** qui dépasse le périmètre de ce rapport.

Références

- **README.md** (racine du dépôt), journal complet des runs, métriques, dépannage et guide de déploiement.
- **docs/00_cadrag/01_cahier_des_charges.md**, cahier des charges de la mission.
- **vllm-project/vllm**, issue #58616, signalement du segfault et de sa résolution.
- **docs/01_environnement/00_guide_installation_environnement.md**, guide d'installation.
- **docs/references/M14 Support de presentation_V3.pdf**, support de présentation détaillé, référence complémentaire.